Exercise 1: Text Embedding and Semantic Similarity Search

Aim

To generate text embeddings for a set of documents and perform semantic similarity search that retrieves the documents most relevant to a user query by meaning, not by exact keywords.

Algorithm

1. Import the required libraries (sentence-transformers, numpy).
2. Load a pre-trained embedding model (all-MiniLM-L6-v2).
3. Define a collection of sample documents.
4. Convert all documents into embedding vectors.
5. Accept a search query and convert it into an embedding vector.
6. Compute the cosine similarity between the query vector and each document vector.
7. Rank the documents by similarity score.
8. Display the top matching results.

Python Code

```
from sentence_transformers import SentenceTransformer
import numpy as np

# Step 1: Load the pre-trained embedding model
model = SentenceTransformer("all-MiniLM-L6-v2")
```

```
# Step 2: Sample document collection
documents = [
    "A smart fitness watch tracks steps, heart rate and sleep.",
    "Wireless earbuds provide clear sound and noise cancellation.",
    "Python is a popular programming language for data science.",
    "Machine learning models learn patterns from large datasets.",
    "A healthy diet and regular exercise improve overall wellbeing.",
    "The stock market closed higher after strong earnings reports.",
]

# Step 3: Generate embeddings for the documents
doc_embeddings = model.encode(documents)
print("Embedding shape:", doc_embeddings.shape)

# Step 4: Cosine similarity function
def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

# Step 5: Semantic search function
def semantic_search(query, top_k=3):
    query_embedding = model.encode(query)
    scores = [cosine_similarity(query_embedding, d) for d in doc_embeddings]
    ranked = np.argsort(scores)[::-1][:top_k]

    print(f"\nQuery: {query}")
    print("-" * 60)
    for rank, idx in enumerate(ranked, start=1):
        print(f"{rank}. [{scores[idx]:.4f}] {documents[idx]}")

# Step 6: Run sample queries
semantic_search("device to monitor my workout and health")
semantic_search("learning to code for AI")
```

Sample Output

```
Query: device to monitor my workout and health
------------------------------------------------------------
1. [0.6405] A smart fitness watch tracks steps, heart rate and sleep.
2. [0.3020] A healthy diet and regular exercise improve overall wellbeing
3. [0.2155] Wireless earbuds provide clear sound and noise cancellation.

Query: learning to code for AI
------------------------------------------------------------
1. [0.3691] Machine learning models learn patterns from large datasets.
2. [0.2267] Python is a popular programming language for data science.
3. [0.0989] A smart fitness watch tracks steps, heart rate and sleep.
```

Result

The Embeddings were generated with a pre-trained sentence-transformer model, and cosine similarity search retrieved the most relevant documents by meaning, not exact keywords